
Préparation au Concours National Commun 2026 — Informatique MP

Épreuve d'Informatique

Filière MP — Durée : 4 heures

Le problème de partition : recherche exhaustive, NP, programmation dynamique et reconstruction

Auteur : **Pr Agrégé El Hadiq Zouhair****cpgeacademy.org** — Tous droits réservés © cpgeacademy.org

Avis important. Ce document est une **initiative personnelle** du professeur, à but strictement pédagogique. Il ne s'agit **pas** d'un sujet officiel du Concours National Commun ; il s'en inspire uniquement par la forme et l'esprit. **Pour aller plus loin.** Les élèves peuvent traiter une **nouvelle version** de ce problème dotée d'un **autocorrecteur des réponses**, et suivre un **cours complet**, sur **cpgeacademy.org**.

Présentation et notations

Le *problème de partition* consiste, étant donné une liste S d'entiers strictement positifs, à décider si l'on peut la séparer en deux sous-ensembles de même somme. Il apparaît dans les preuves à divulgation nulle de connaissance (chapitre « Zero-Knowledge Proofs » de *Mastering Ethereum*) comme exemple canonique de problème *facile à vérifier* mais *difficile à résoudre*. On note $T = \sum_{x \in S} x$ la somme totale.

Conventions. Langage Python 3 ; **sum**(S) est la somme des éléments, **len**(S) leur nombre. Le module `itertools` fournit `combinations( $S$ ,  $r$ )` qui énumère les sous-ensembles de S à r éléments. Questions numérotées **en continu**.

ÉNONCÉ

Partie I — Recherche exhaustive

Question 1. Écrire `demi_somme( $S$ )` qui renvoie `None` si $T = \text{sum}(S)$ est impair, et $T/2$ (entier) sinon.

Question 2. Écrire `partition_exhaustive( $S$ )` qui décide, en énumérant tous les sous-ensembles à l'aide de `itertools.combinations`, s'il existe un sous-ensemble de somme $T/2$. La fonction renvoie un booléen. On renverra `False` si T est impair.

Question 3. Combien de sous-ensembles possède un ensemble à n éléments ? En déduire que `partition_exhaustive` a une complexité temporelle en $\Theta(2^n)$ dans le pire des cas, et expliquer pourquoi cette approche est inutilisable pour n grand.

Partie II — Le problème de décision Subset-Sum

On suppose désormais T paire et on pose $C = T/2$. Le *problème de décision* Subset-Sum demande : « existe-t-il un sous-ensemble de S de somme exactement C ? »

Question 4. Décrire un *certificat* (une preuve courte) permettant de convaincre en temps polynomial qu'une instance est positive, et l'algorithme de *vérification* associé. Donner sa complexité.

Question 5. En déduire que Subset-Sum appartient à la classe **NP**. On rappelle qu'un problème de décision est dans *NP* s'il admet un certificat vérifiable en temps polynomial. On mentionnera, sans le démontrer, que Subset-Sum est en fait NP-complet.

Partie III — Récurrence et table de programmation dynamique

On définit, pour $0 \leq i \leq n$ (avec $n = \text{len}(S)$) et $0 \leq c \leq C$, le booléen

$A[i][c]$ = « il existe un sous-ensemble des i premiers éléments de S de somme c ».

Question 6. Justifier la relation de récurrence

$$A[i][c] = A[i-1][c] \vee (c \geq S[i-1] \wedge A[i-1][c - S[i-1]]) \quad (i \geq 1),$$

et préciser l'initialisation $A[0][\cdot]$.

Question 7. En déduire une fonction `subset_sum_2d(S, C)` qui construit la table A (de taille $(n+1) \times (C+1)$) et renvoie $A[n][C]$. Donner sa complexité en temps et en espace.

Partie IV — Version optimisée en espace

Question 8. Écrire `peut_partitionner(S)` qui, après avoir écarté le cas T impair, décide l'existence d'une partition équilibrée en n'utilisant qu'un tableau **dp à une dimension** de taille $C+1$. *Indication* : parcourir c de C jusqu'à x en **décroissant** pour chaque élément x de S .

Question 9. Expliquer précisément pourquoi le parcours **décroissant** de c est nécessaire : que se passerait-il avec un parcours croissant ?

Question 10. Donner la complexité en temps et en espace de `peut_partitionner`, et expliquer le terme *pseudo-polynomial* (comparer C à la taille de codage de l'entrée).

Partie V — Reconstruction et correction

Question 11. À partir de la table A de la Partie III, écrire `sous_ensemble(S, C)` qui renvoie un sous-ensemble de somme C (sous forme de liste), ou `None` s'il n'en existe pas. *Indication* : remonter la table de $i = n$ à 1 ; l'élément $S[i-1]$ est pris ssi $A[i-1][C]$ est faux (le retirer impose de l'utiliser).

Question 12. Énoncer un *invariant* portant sur `dp` après le traitement des i premiers éléments dans `peut_partitionner`, puis le démontrer par récurrence sur i . Conclure sur la correction, et justifier la terminaison.

CORRIGÉ

Partie I — Recherche exhaustive

Question 1.

```

1 def demi_somme(S):
2     T = sum(S)
3     if T % 2 == 1:
4         return None
5     return T // 2

```

Question 2.

```

1 import itertools
2
3 def partition_exhaustive(S):
4     T = sum(S)
5     if T % 2 == 1:
6         return False
7     C = T // 2
8     for r in range(len(S) + 1):
9         for comb in itertools.combinations(S, r):
10             if sum(comb) == C:
11                 return True
12     return False

```

Question 3. Un ensemble à n éléments possède 2^n sous-ensembles (chaque élément est pris ou non). La boucle les parcourt tous dans le pire des cas (instance négative), et chaque test de somme coûte $O(n)$: la complexité est $\Theta(n 2^n) = \Theta(2^n)$ à un facteur polynomial près. Pour $n = 60$, $2^{60} \approx 10^{18}$: irréalisable. $\square$

Partie II — Le problème de décision Subset-Sum

Question 4. Le certificat est la *liste des indices* d'un sous-ensemble candidat $I \subseteq \{0, \dots, n-1\}$. La vérification calcule $\sum_{k \in I} S[k]$ et teste l'égalité à C ; elle lit chaque indice une fois, d'où une complexité $O(n)$, polynomiale. Le certificat a une taille $O(n)$, elle aussi polynomiale.

Question 5. Puisque toute instance positive admet un certificat de taille polynomiale vérifiable en temps polynomial, Subset-Sum est dans **NP**. (On admet qu'il est de plus NP-complet, donc parmi les plus difficiles de NP.) $\square$

Partie III — Récurrence et table

Question 6. Un sous-ensemble des i premiers éléments de somme c soit *n'utilise pas* $S[i-1]$ (c'est alors un sous-ensemble des $i-1$ premiers de somme c , d'où $A[i-1][c]$), soit *l'utilise* (c'est alors un sous-ensemble des $i-1$ premiers de somme $c - S[i-1]$, licite seulement si $c \geq S[i-1]$, d'où $A[i-1][c - S[i-1]]$). D'où $A[i][c] = A[i-1][c] \vee (c \geq S[i-1] \wedge A[i-1][c - S[i-1]])$. Initialisation : avec zéro élément, seule la somme 0 est atteignable, donc $A[0][0] = \text{vrai}$ et $A[0][c] = \text{faux}$ pour $c \geq 1$. $\square$

Question 7.

```

1 def subset_sum_2d(S, C):
2     n = len(S)
3     A = [[False] * (C + 1) for _ in range(n + 1)]
4     for i in range(n + 1):
5         A[i][0] = True # somme 0 : sous-ensemble vide
6     for i in range(1, n + 1):
7         x = S[i - 1]
8         for c in range(C + 1):
9             A[i][c] = A[i-1][c] or (c >= x and A[i-1][c - x])
10    return A[n][C]

```

Complexité : $O(nC)$ en temps (deux boucles imbriquées) et $O(nC)$ en espace (table complète).

Partie IV — Version optimisée en espace

Question 8.

```

1 def peut_partitionner(S):
2     T = sum(S)
3     if T % 2 == 1:
4         return False
5     C = T // 2
6     dp = [False] * (C + 1)
7     dp[0] = True
8     for x in S:
9         for c in range(C, x - 1, -1): # DECROISSANT
10            if dp[c - x]:
11                dp[c] = True
12    return dp[C]

```

Question 9. Le tableau dp représente la ligne courante $A[i - 1][\cdot]$ que l'on met à jour en $A[i][\cdot]$ « sur place ». En parcourant c de façon *décroissante*, la case $dp[c - x]$ lue n'a pas encore été modifiée à ce tour : elle vaut encore $A[i - 1][c - x]$, ce qui est correct. Avec un parcours *croissant*, $dp[c - x]$ aurait déjà pu passer à **True** *au même tour*, ce qui reviendrait à utiliser x **plusieurs fois** (comme dans un sac à dos avec répétitions) : on résoudrait un autre problème et la réponse serait fausse.

Question 10. Complexité : $O(nC)$ en temps, $O(C)$ en espace. L'entrée se code sur $O(\sum_i \log S_i)$ bits ; or C peut être de l'ordre de $\max_i S_i$, donc *exponentiel* en le nombre de bits de codage. L'algorithme est polynomial en la *valeur* C mais non en la *taille* de l'entrée : on dit qu'il est *pseudo-polynomial*. $\square$

Partie V — Reconstruction et correction

Question 11.

```

1 def sous_ensemble(S, C):
2     n = len(S)
3     A = [[False] * (C + 1) for _ in range(n + 1)]
4     for i in range(n + 1):
5         A[i][0] = True
6     for i in range(1, n + 1):
7         x = S[i - 1]
8         for c in range(C + 1):
9             A[i][c] = A[i-1][c] or (c >= x and A[i-1][c - x])
10    if not A[n][C]:

```

```

11     return None
12     res, c = [], C
13     for i in range(n, 0, -1):
14         if not A[i-1][c]: # S[i-1] est indispensable
15             res.append(S[i-1])
16             c -= S[i-1]
17     return res[::-1]

```

Vérification : `sous_ensemble([1, 1, 5, 7], 7) = [1, 1, 5]` (somme 7).

Question 12. Invariant. Après le traitement des i premiers éléments de S dans `peut_partitionner`, pour tout $0 \leq c \leq C$, `dp[c]` est vrai si et seulement s'il existe un sous-ensemble des i premiers éléments de somme c (c.-à-d. `dp[c] = A[i][c]`).

Base ($i = 0$) : seul `dp[0]` est vrai, correspondant au sous-ensemble vide : conforme à `A[0]`.

Hérédité : on suppose `dp = A[i-1][·]` et on traite $x = S[i-1]$. Pour chaque c (de C à x), on met `dp[c]` à `True` si `dp[c-x]` est vrai. Le parcours décroissant garantit `dp[c-x] = A[i-1][c-x]` ; combiné à la valeur préexistante `A[i-1][c]` de `dp[c]`, on obtient exactement `A[i][c] = A[i-1][c] ∨ (c ≥ x ∧ A[i-1][c-x])`. L'invariant est donc préservé. *Conclusion* : après les n éléments, `dp[C] = A[n][C]`, vrai ssi une partition équilibrée existe : la fonction est correcte. **Terminaison** : toutes les boucles `for` portent sur des intervalles finis. $\square$

Vérifications numériques (Python) : sur 2000 listes aléatoires, `peut_partitionner` et `partition_exhaustive` renvoient *toujours* le même résultat. Exemples : `[1, 1, 5, 7] ↦ True`, `[1, 1, 5, 8] ↦ False`, `[3, 3, 3, 3] ↦ True`.